

字符串算法

字符串是信息学竞赛中比较重要的一个模块，在普及组，提高组，省选，NOI比赛中均有出现。比较简单的题主要考察字符串模拟，稍微复杂一点的题就会涉及到一些字符串算法，一些更难的题还会用到字符串算法+其他算法，比如字符串+动态规划，字符串+数据结构等等。

同时，无论是字符串算法还是其他算法，都是对思维的锻炼，在很多地方都有共通之处，很多时候也会结合在一起。比如有的时候，虽然题目中是数字，但是要转换成字符串来处理，有的题目虽然是字符串，但是要转换成数字来处理。



字符及字符串



字符

我们都知道，所有信息在计算机中都是以二进制的形式存储的，同样，字符也是。字符在计算机中是以ASCII码的形式存在，ASCII码的本质就是数字。

也就是说，一个字符有一个唯一的ASCII码与之对应，同样，一个ASCII码也对应着一个固定的字符，他们之间存在一一映射的关系。

很多时候，因为计算需要，需要对字符(字符串)和数字进行相互转换来达到目的。比如有的题目中信息全是数字，但是却需要把数字当做字符串来处理，有的题目中看上去是对字符串进行求解，但是却需要先把字符串转换成数字做来提高效率。

ASCII表

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位 低四位		ASCII控制字符											ASCII打印字符														
		0000						0001					0010		0011		0100		0101		0110		0111				
		0						1					2		3		4		5		6		7				
		十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl	
0000	0	0		^@	NUL	\0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p		
0001	1	1	☺	^A	SOH		标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q		
0010	2	2	☹	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r		
0011	3	3	♥	^C	ETX		正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s		
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t		
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u		
0110	6	6	♠	^F	ACK		肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v		
0111	7	7	●	^G	BEL	\a	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w		
1000	8	8	◼	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x		
1001	9	9	◯	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y		
1010	A	10	◼	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z		
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{		
1100	C	12	♀	^L	FF	\f	换页	28	└	^\ FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124				
1101	D	13	♪	^M	CR	\r	回车	29	↔	^] GS		组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}			
1110	E	14	🎵	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~		
1111	F	15	🎵	^O	SI		移入	31	▼	^_ US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	^Backspace 代码: DEL		

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。

2013/08/08

ASCII和字符的转换

问题一：如果我要知道某个字符对应的ASCII怎么办？

```
char a;  
scanf("%c",&a);    printf("%d",a);
```

如果输入字符，当输出格式是%c时表示输出字符，如果输出格式是%d表示输出该字符对应的ASCII值。

问题二：我怎么知道输入的数字是字符还是数字？

```
char a;                int a;  
scanf("%c",&a);        scanf("%d",&a);  
char a='1';           int a=1;
```

要判断输入的是ASCII值还是字符，主要看输入格式，%d表示输入的ASCII值，%c表示输入的字符。字符用' '（单引号）括起来。

字符读入输出

在涉及到字符以及字符串读入的时候，已经要注意格式，因为“空格”“回车”都属于字符，所以一定要去除一些无关操作的影响。

(1) 数字 字符

```
scanf("%d %c", &a, &b);
```

(2) 第一行一个数字，后面n行，每行一个字符+数字

```
scanf("%d", &n);  
for(int i=1; i<=n; i++)  
scanf("\n%c %d", &a, &b);
```

如果空格或者回车之后是数字的，就不需要考虑，因为空格和回车不是数字，是不会误判的，当然你也可以用%s读入一个字符，这样也不需要考虑空格了。

判断是否读入错误的办法很简单，把你的输入输出一遍就可以了，比如%d...%c...\n的形式。

字符串

存储多个字符的数组被称为字符数组，也可以称为字符串。

定义：char s[100];

输入：

第一种：

```
scanf("%s", s);
```

第二种：

```
for (int i=0; i<n; i++)  
scanf("%c", &s[i]);
```

区别：

- 1: 字符串的整体输入用%s，并且输入不用"&"，而字符的单个输入要用"&"。
- 2: 字符串可以整体赋值也可以单个赋值，单个赋值用%c,循环赋值
- 3: %s输入字符串的下标是从0开始。如果不知道字符串到底有长，只能用第一种输入。

输入

第一种：

```
scanf ("%s", s);
```

第二种：

```
for (int i=0; i<n; i++)  
scanf ("%c", &s[i]);
```

区别：

- 1：字符串的整体输入用%s，并且输入不用“&”，输入直接输入数组名而不需要数组下标，表示整个数组。而字符的单个输入要用“&”，需要数组下标，表示单个元素。
- 2：字符串可以整体赋值也可以单个赋值，单个赋值用%c表示单个元素，使用循环赋值。
- 3：%s输入字符串的下标只能是从0开始。
- 4：如果不知道字符串到底有多长，只能用第一种输入。
- 5：如果字符串中含有空格，只能用第二种输入。

输出

第一种：

```
printf("%s", s);
```

第二种：

```
for (int i=0; i<n; i++)  
printf("%c", &s[i]);
```

输出格式和输入一样，最好输入输出格式匹配，以免因为下标不一样造成不必要的麻烦。

一：第一种输出仍然是从下标为0的元素开始输出。

为什么第一种输入不能包括空格：

因为scanf函数当遇到空格或者回车时表示结束。

```
scanf("%s", s);
```

如果输入hello world，实际上读入数组s的只有hello，空格表示读入结束

数组赋值

```
char s[100]={'h','e','l','l','o'}; ✓
```

```
char s[100]= "hello"; ✓
```

```
char s[100];  
s="hello"; ✗
```

```
char s[100];  
s[0]='h';  
s[1]='e';  
s[2]='l';  
s[3]='l';  
s[4]='o'; ✓
```

字符用单引号，字符数组用双引号。字符串定义之后不能直接整体赋值，定义之后再赋值只能单个循环赋值

字符串函数

复制函数：

strcpy(字符串名1, 字符串名2)：

将字符串2**完全**复制给字符串1.

```
char a[100], b[100];
```

```
strcpy(a,b);
```

strncpy(字符串名1, 字符串名2, 长度n)：

将字符串2中前n个字符复制给字符串1.

```
char a[100], b[100];
```

```
strncpy(a,b,5);
```

字符串函数

比较函数：

strcmp(字符串名1, 字符串名2)：

比较两个字符串的大小(ASCII)，从第一个字符开始比较，如果字符串1>字符串2, 结果返回正整数，相等则返回0，小于则返回负整数。

```
char a[100]="apple";   char b[100]="Apple";  
if(strcmp(a,b)>0)      ...  
else if(strcmp(a,b)=0) ...  
else if(strcpy(a,b)<0)  ...
```

strncmp(字符串名1, 字符串名2, n)：

比较两个字符串前n个字符的大小关系。返回值和上面一样

字符串函数

其他函数：

strlen(字符串名)：

计算字符串的长度，返回值为字符串的长度，需要
用一个变量来保存

```
printf("%s",a);  l=strlen(a);
```

strlwr(字符串名)：

把字符串中所有大写字母转换成小写字母。

strupr(字符串名)：

把字符串中所有小写字母转换成大写字母。

**注意：所有的字符串函数的使用都需要
新增头文件：#include<cstring>**



字符串算法





字符串判等

给定两个字符串 a, b ，判断这两个字符串是否相等



字符串匹配

给定两个字符串a, b, 判断字符串b是否是字符串a的子串。

子串：字符串中任意个连续的字符组成的子序列
比如abc的字符串有：a, b, c, ab, bc, abc

子集： 无序，不连续

子序列：有序，不连续

子串： 有序，且连续

字符串匹配

a串



b串



很容易想到一种匹配方法，先让a串第一个字符和b串第一个字符匹配，如果相等，再让a串第二个字符和b串第二个字符串匹配，如果相等，继续循环执行上一步操作。

但是如果匹配到某个位置无法匹配了，该怎么处理呢？

字符串匹配

a串



b串



如果不可以匹配，表示b串和以a串开头的这个子串不匹配，那么我们就需要继续去枚举a串的子串，既然以a串开头的子串不可以匹配，那么我们就让b串和以a串下一个字符开头的子串进行匹配。匹配方式和前面相同

a串



b串



字符串匹配

a串



b串



a串



b串



如果匹配仍然失败，那么我们继续让a串的开头字符后移，形成新的子串，直到a，b串匹配成功为止，或者所有的a串的子串都尝试过，都不能与b串匹配就结束了。

字符串匹配

```
for(int i=0;i<=la-lb;i++)//枚举a的子串起始位置
{
    cnt=0;
    for(int j=0;j<lb;j++)//一次完整的匹配
    {
        if(a[i+j]==b[j])
            cnt++;
        else
            break;
    }
    if(cnt==lb)
    {
        printf("yes");
        return 0;
    }
}
```

设a串长度为n，b串长度为m，那么该算法的时间复杂度为 $O(n*m)$

字符串匹配

前面的匹配方法效率特别低，是一种字符串的暴力匹配算法，效率低的原因是一个一个枚举字符串的子串，但是很多时候并不需要逐个枚举，很多枚举都是无效的。比如：

abcabe

abe

aaaaaaaaaaaaaab

aaaaaab

我们会发现，当第一次匹配失败的时候，很多时候并不需要一个一个往后移，那么我们能找到一个具体的应该移到的位置吗？

字符串匹配

a串



b串

j: 最后一个匹配的字符

a串



b串

i: 下一次匹配的开始位置

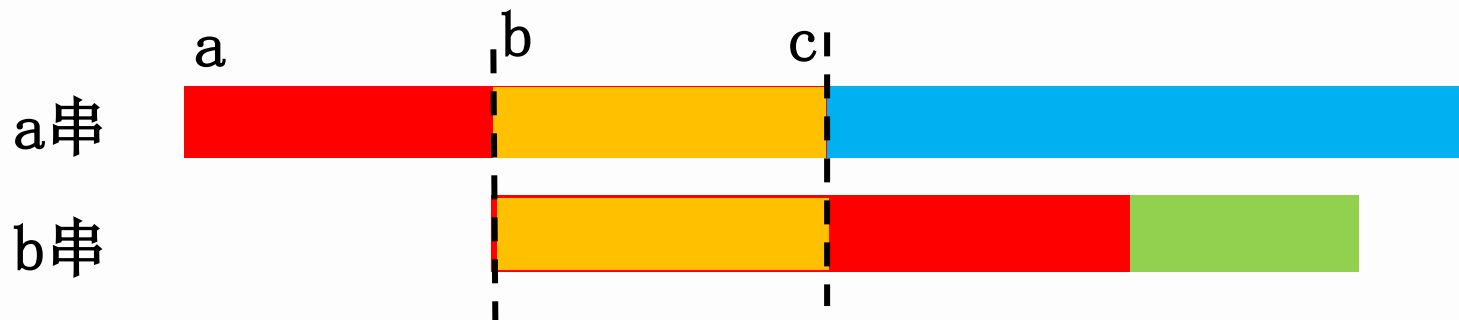
a串



b串

就必须保证 $i-j$ 这一段是相等的

字符串匹配



a: 第一次匹配的起始位置

a—c: 上一次已经正确匹配到长度

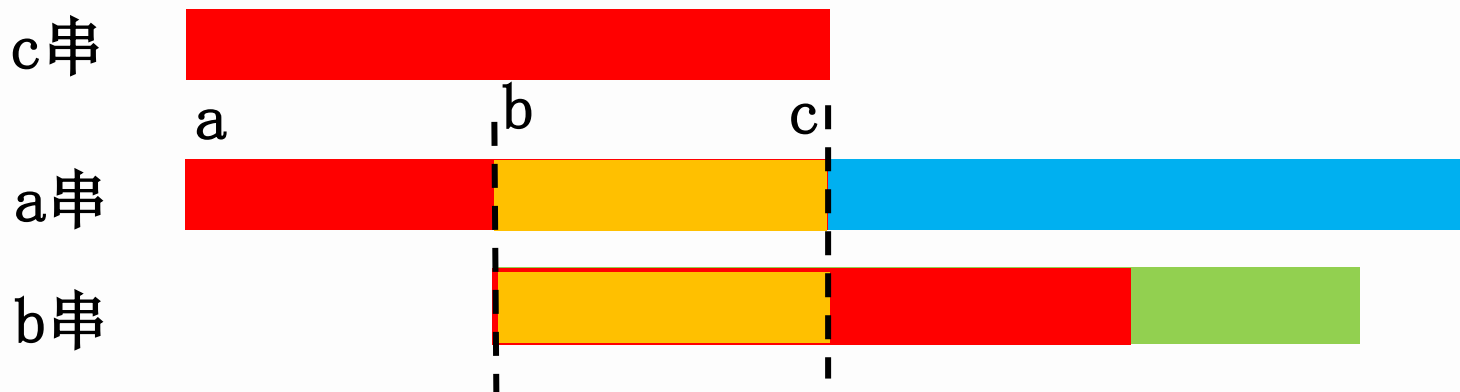
b: 下一次匹配应该跳到的位置

b-c: 下一次匹配既然是跳到b, 那么说明b—c这一段肯定是可以匹配的, 至于c之后, 因为还未比较过, 所以不清楚

d之后: 等待匹配的长度

a, b, c三个位置, a, c都是已知的, 而我们需要做的就是求出b, 接下来我们来研究一下b—c这一段的意义。

字符串匹配



我们设上一次已经正确匹配过的串为c串，那么对于a串而言，黄色部分(b--c)是c串的后缀，对于b串而言，黄色部分(b--c)是串的前缀，所以黄色部分既是c串的前缀又是c串的后缀，即黄色部分是c串的公共前后缀。

所以只要对于a字符串的每一个位置，我们都知道从开始位置到当前位置的公共前后缀的长度，那么我们就可以直接得到b的位置，就可以快速进行下一次匹配。

字符串匹配

a串



b串



下一次匹配



是否存在？？？

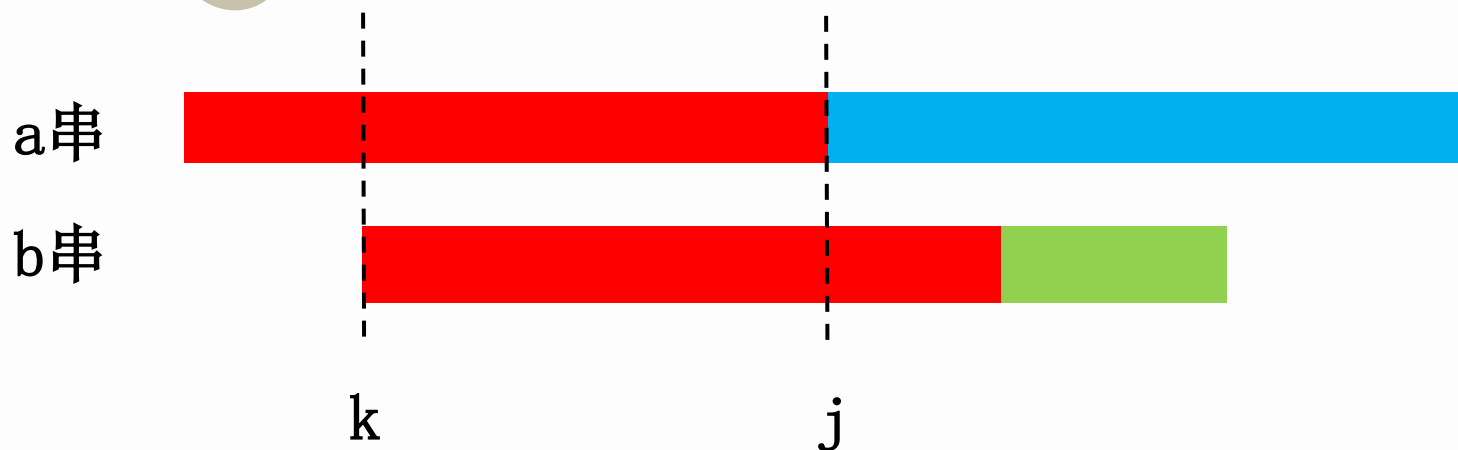


黄色部分：公共前后缀

红色部分：a串和b串相等部分

那么下一次匹配的位置直接移到公共前后缀的后缀所在位置肯定是正确的，因为后面肯定是可以匹配的。
那么前面是否一定不存在可以匹配的位置呢？

字符串匹配



假设在前面存在一个位置 k ，也能保证 k — j 这一段两个字符串相等，那么 k — j 这一部分肯定必须是红色已经匹配部分的公共前后缀，因为后面蓝色部分未知，所以这种情况也可能就是正确匹配方式。

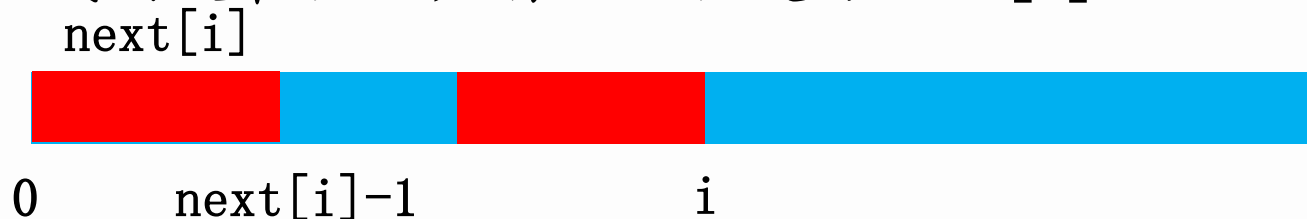
但是一个字符串的公共前后缀可能不止一个，比如aaaaa，公共前后缀可以是a, aa, aaa, aaaa（公共前后缀不能是本身，否则没有意义），那么这些位置都是可以匹配的，所以我们都不能漏掉，即我们应该找的是最大公共前后缀

字符串匹配

那么字符串中任意一个位置的最长公共前后缀应该如何得到呢？如果是对于每一个位置，再去枚举计算出来是很麻烦的，既然每一个位置都需要得到最长公共前后缀，我们不妨考虑一下递推。

假设我知道第 i 个位置的最长公共前后缀，我该如何求解第 $i+1$ 个位置的最长公共前后缀？

我们设最长公共前后缀长度为 $\text{next}[i]$ 。



既然 i 的最长公共前后缀是 $\text{next}[i]$ ，那么说明 $[0, \text{next}[i]-1] = [i-\text{next}[i]+1, i]$ ，那么如果两个区间的后一个位置的字符也相等，即 $a[\text{next}[i]] = a[i+1]$ 那么，很明显 $\text{next}[i+1] = \text{next}[i] + 1$

字符串匹配

但是如果第 $\text{next}[i]$ 的位置与第 $i+1$ 个位置的字符不相等呢？那就是0吗？其实并不是。

比如：abaabab

当我们算出来abaaba的最长公共前后缀是3(aba)时，我们接下来要算abaabab的最长公共前后缀，虽然低4个字符是a和b不相等，但是我们并不能说abaabab的最长公共前后缀就是0。

a串

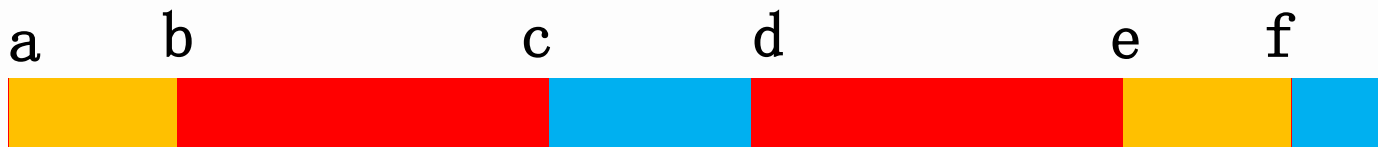


a串



因为 i 的任意一个公共前后缀都可以让上一个串的末尾几个字符=下一个串的起始几个字符，都是有可能的。

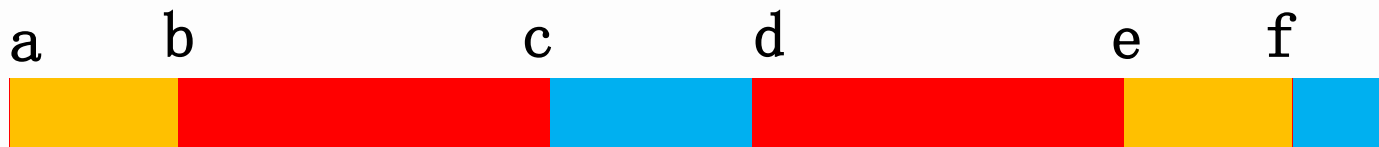
字符串匹配



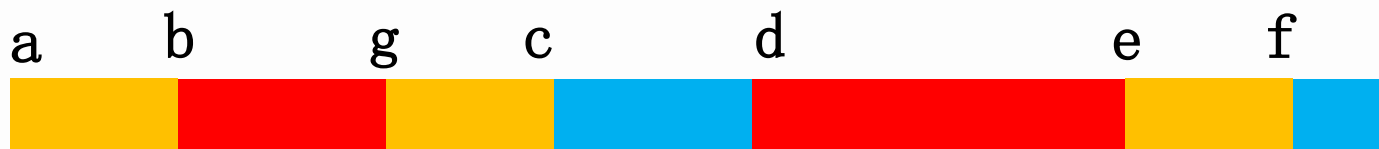
a—c和d—f是最长公共前后缀，但是下一次匹配失败了，但是a—b也是其中的一个公共前后缀，我们如果比较b的下一个字符和f的下一个字符，如果可以匹配，说明f的下一个字符的最长公共前后缀就是a—b长度+1，如果不可以匹配呢？我们是不是还可以继续去找是否存在更短的公共前后缀，直到不存在更短的公共前后缀为止。

所以我们应该依次从大到小枚举公共前后缀，直到找到可以匹配的位置或者所有公共前后缀都不可以匹配为止。但是我们应该如何实现呢？

字符串匹配

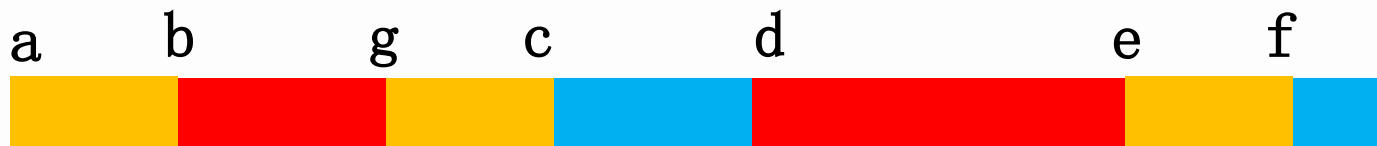


我们继续观察一下图中，e—f除了和a—b相等，还和哪一段相等。



我们会发现e—f和g—c是相等，那么a—b和g—c也是相等的，那么a—b这一段实际上就是a—c的最长公共前后缀。

字符串匹配

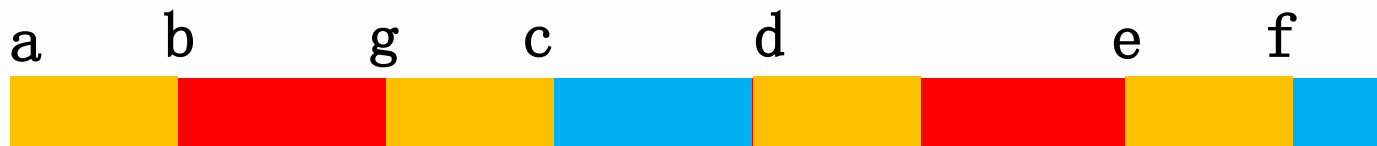


假设 $a \sim c$ 的最长公共前后缀是 $a \sim b$ ，那么 $a \sim c$ 和 $a \sim b$ 之间是否存在一个前后缀被遗漏了呢？

并不会存在，因为如果存在一个前后缀 $a \sim b'$ 长度比 $a \sim b$ 长，那么画图就可以发现 $a \sim b$ 不是 $a \sim c$ 的最长公共前后缀，和我们的假设矛盾，所以我们可以不断去找最长公共前后缀来求解下一个位置的最长公共前后缀。

所以求解公共前后缀方法的有了。

字符串匹配



已知 i 的最长公共前后缀为 $\text{next}[i]$ ，那么 $i+1$ 的最长公共前后缀：

(1) $a[i+1] = a[\text{next}[i]]$ $\text{next}[i+1] = \text{next}[i] + 1$

(2) 如果不相等，我们设 $j = \text{next}[i] - 1$

那么我们去比较 $a[i+1] = a[\text{next}[j]]$

如果可以匹配，那么 $\text{next}[i+1] = \text{next}[j] + 1$

如果不可以匹配呢？那么继续让 $k = \text{next}[j] - 1$

再去比较 $a[i+1] = a[\text{next}[k]]$

直到 $k=0$ 或者可以匹配为止。

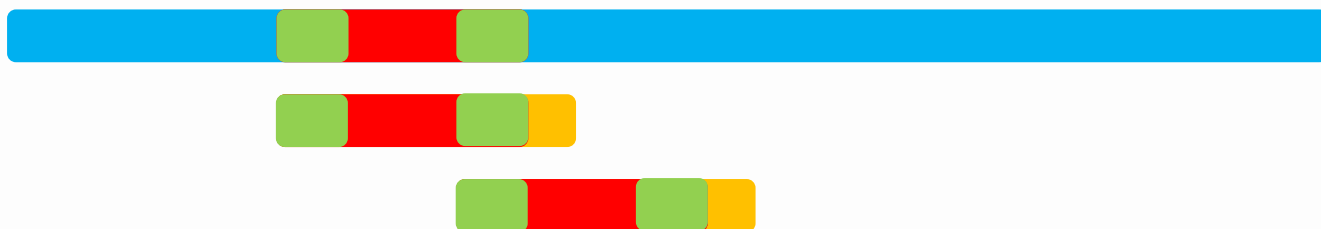
字符串匹配

```
nextn[0]=0; // 第一个字符的最长公共前后缀肯定是0
int j=0; // j表示当前匹配的最长公共前后缀的长度
for(int i=1; i<la; i++)
{
    while(a[i]!=a[j]&& j)
    {
        j=nextn[j-1]; // 得到的j一直是长度
    }
    if(a[i]==a[j]) // 如果都不匹配, j=0
        j++; // 公共前后缀+1
    nextn[i]=j;
}
// 注意, next是关键字, 不能开做全局变量, 最好不做
```

因为不存在重复的匹配, 所以总体的时间复杂度为 $O(n)$

KMP算法

关键是最长公共前后缀有什么用呢？
我们来模拟一下匹配过程。



当继续匹配失败的时候(黄色和蓝色部分)，常规做法就是把下面这个字符串向后面挪动一位，重新匹配。但是其实我们可以直接挪到下一个位置，即直接比较已经匹配的最后位置的next指针的位置。

证明很好证明，反证，如果前面还存在相等的位置。。。

KMP算法

```
int j=0;
for(int i=0;i<l1;i++)
{
    while(s1[i]!=s2[j]&&j)
    {
        j=nexts[j-1];
    }
    if(s1[i]==s2[j])
    j++;
    if(j==l2)
    printf("%d\n",i-l2+2);
}
```

j 表示当前需要匹配的位置，如果不可以匹配，应该去找前一个可以匹配的next指针($j-1$), $next[j-1]$ 表示最后一个可以匹配的位置，那么应该继续比较下一个位置为 $next[j-1]-1+1$.



练习题

- 1: 洛谷 3375: KMP模板
 - 2: HDU 2087: 剪花布条
 - 3: 洛谷 2580: 于是他开始错误的点名了
 - 4: Poj 2752 Seek the Name
 - 5: 洛谷 2292 L语言
 - 6: 洛谷 2375 动物园
- 



Power string

给定若干个长度小于等于1000000000的字符串，询问每个字符串最多由多少个相同的子串重复连接而成。如ababab最多由3个ab连接而成。

输入：

abcd

aaaa

ababab

...

输出：

1

4

3



KMP求最小循环节

很容易想到第一种思路是先按照位数分解质因数，然后再用hash值来判断是否是循环节，但是这样做效率不是很高，涉及到质因数分解以及快速幂，总的时间复杂度会多一个log，而且hash验证的时候比较麻烦，所以我们考虑另一种解法。

在学习KMP的时候，我们理解了next数组，也就是公共前后缀。我们来看几个有循环节的字符串：

ababab:next[6]=4 循环节长度为2

abcabc:next[6]=3 循环节长度为3

ababaab:next[7]=2 无循环节

aaaaaa:next[6]=5 循环节长度为1

好像有点规律，我们再画图看一下。

KMP求最小循环节

字符串长度为a，最后一个字符的next数组的值为b。
根据图像很容易看出来如果两端覆盖没有交叉，肯定无循环节。

我们假设一个字符串有循环节，循环节长度为b，
那么很明显，最后一个字符串的next数组的长度为a-b。

反过来，我们知道字符串字符串的长度和next数组，是可以直接求出循环节的，但是，我们要如何判断是否有循环节呢？很明显，既然是循环节，那么肯定要整除。所以我们只需要判断

$$1 \% (1 - \text{next}[1-1]) == 0 \text{ 就可以了。}$$

BZOJ 1355

给定一个字符串的子串，已知这个原字符串是由某个字符串不断自我连接形成的，问该字符串的最短长度是多少。

输入：

8

cabcabca

输出：

3

他可以由cab或者bca连接而成，比如三个cab连接而成cabcabcab，给定的字符串cabcabca是他的子串。

BZOJ 1355

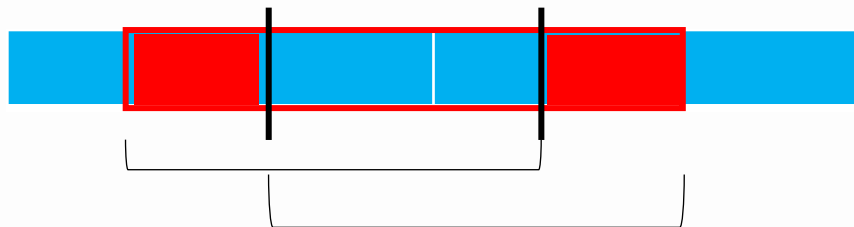
该题的本质还是求最小循环节。先由循环节多次循环组成一个字符串，然后再从中截取一段字符串出来。

我们先假设原字符串的最短循环节长度为 k

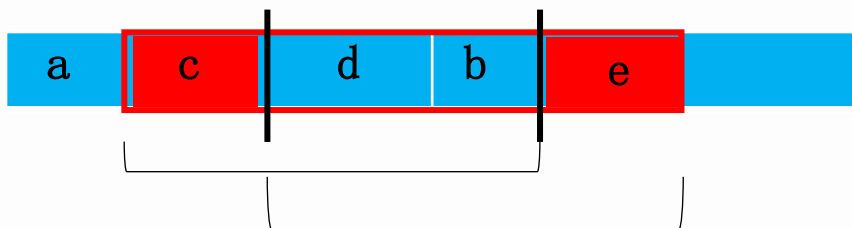


红色部分为题目中给出的字符串的子串。

既然 k 是最短循环节，那么给定的字符串的子串长度肯定大于等于 k 。如果给定字符串的子串长度 $< k$ ，那么直接把该子串当做循环节就是答案了。



BZOJ 1355



我们会发现，红色部分是相同的，也就是说给定的字符串是存在公共前后缀的，那么我们再观察一下循环节和给定字符串长度之间的关系。

我们会发现给定字符串子串==原字符串的最小循环节+e，而e代表的含义就是给定字符串子串的最长公共前后缀。所以该题和前一道题是一样的做法，先求出给定字符串子串的最长公共前后缀，再用总长度减去最长公共前后缀的长度就是正确答案了。

BZOJ 1355

其实该题还是用kmp求循环节，最短循环节的长度仍然是 $l - \text{next}[l-1]$ ，画一个图就出来了，是他的子串不影响原来的求法。



如果是前面残缺了一部分和后面残缺一部分是相同的。

BZOJ 1511

先定义字符串的周期：对于两个字符A,B，如果A是B的前缀，并且B是AA的前缀，那么A就是B的周期，如果不存在，周期就是0，比如abab和ababab都是abababa的周期，ababab的最大周期是abab，先给出一个串，求出他所有前缀的最大周期之和。

输入：

8

babababa

输出：

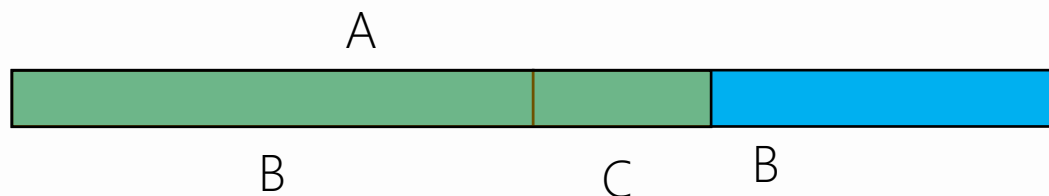
24

$$0+0+2+2+4+4+6+6=24$$

$$n \leq 1000000$$

BZOJ 1511

一般情况下，只要涉及到前后缀的问题大部分都可以用kmp来求解，对于可以用kmp求解的问题，如果以前没有遇到过或者不是很能理解，那么最好的办法就是画图。



画图之后我们很容易就看出来对于A串来说，只需要A有公共前后缀就可以了(C)，这就满足了B是A的前缀，同时A是BB的前缀，该题要求最大周期，也就是最大的B，很明显要C最短，实际上就是已知求该位置的next数组，直到最后一个不为零的位置为止。也就是说该题是求最小公共前后缀(如果不存在周期就是0，存在就是最小的不为零的那个)

BZOJ 1511

最长公共前后缀我们已经学会怎么求了？那么最短呢？
有一种很暴力的求解方法，对于每个位置的最长公共前后缀，一直找到其next数组指向的位置，直到跳到最后一个不为零的位置为止。



比如ababab, 最后一个字符的最长公共前后缀为4, 那么我们就跳到abab(4), 此时的最长公共前后缀为2, 那么我们就跳到ab(2), 此时最长公共前后缀为0, 那么2就是最小公共前后缀, 所以最大周期就是总长度-最下公共前后缀=6-2=4.

但是如果给定的字符串是形如aaaaaa...aaaa.. 这种呢？
每次的next就是后一个位置-1, 那么每次求解最坏时间复杂度可以到达 $O(n)$

BZOJ 1511

那么我们是否可以直接求解最短公共前后缀呢？

我们还是来找一下第 i 个位置的最短公共前后缀和前面得到的最短公共前后缀的关系。



我们设最短公共前后缀为 $\text{num}[i]$ ，那么第 i 个位置的最短公共前后缀又有哪些可能呢？

如果 i 这个位置不存在最长公共前后缀，那么明显 $\text{num}[i]=0$ ；

如果存在最长公共前后缀，那么第 i 个位置的 num 值实际上就等于 $\text{next}[i]-1$ 这个位置的 num 值，画图可以知道，但是如果 $\text{next}[i]-1$ 的 num 值为0，但是第 i 个位置明显是有公共前后缀的，那么 $\text{num}[i]=\text{next}[i]$.

公共前后缀

求一个字符串的所有前缀的不超过长度的一半的最长公共前后缀。

比如abababab

那么前缀的最长公共前后缀分别是

0

0

1

2

1 (aba超过了长度的一半，所以不算)

2 (abab超过了一半)

3

4

公共前后缀

该题仍然是求最长公共前后缀，但是却有一个限制条件，那就是不能超过前缀长度的一半，有一种很简单的办法，就是一直找最大公共前后缀，直到找到的最大公共前后缀 $\leq$ 长度的一半为止。

但是这样做，还是有个地方有问题，就是效率，如果字符串大部分字符都是相等的，那么你找很多次才能找到符合条件的公共前后缀，时间复杂度会很高。

所以我们要优化一下，同样，我们还是考虑是否可以直接在求解最长公共前后缀的时候就把符合条件的公共前后缀的长度也求出来。

公共前后缀

我们设不超过一半的最长公共前后缀的长度为 $\text{num}[i]$, 对于第 i 个位置的 $\text{num}[i]$, 我们观察一下和前面的 $\text{num}[i]$ 之间存在什么关系?

我们再求解 $\text{nextn}[i]$ 的时候, 如果得到的 $\text{nextn}[i]$ 的值 $\leq$ 长度的一半, 那么 $\text{num}[i] = \text{nextn}[i]$.

但是如果大于一半呢? 那么让 $\text{num}[i] = \text{num}[\text{nextn}[i] - 1]$? 大部分时候是, 但是这样说是错误的, 比如 `aaaa`, $\text{num}[2] = 1$, $\text{nextn}[3] = 3 > (1+3)/2$, 那么 $\text{num}[3] = \text{num}[2] = 1$ 吗? 并不是, 因为这里分奇偶, 所以我们要换一种思路。我们考虑一下第 i 个和第 $i+1$ 个的关系, 如果第 i 个的 num 值为 k , 那么第 $i+1$ 个最多为 $k+1$, 否则就要到 `next` 数组上去。我们这里只需要保证 j 一直不超过长度的一半就可以了。

公共前后缀

```
int j=0;
for(int i=0;i<len;i++)
{
    while(j&& s[j]!=s[i]) j=next[j-1];
    if(s[j]==s[i]) j++;
    if((j<<1)>i+1)//因为上一次j<=(i+1)/2
    //那么下一次i增加了1, 所以j最多也就刚好超过1/2
    //所以最多执行一个next跳转他就会回到1/2的范围内
    j=next[j-1];
    num[i]=j;
}
```

比如aaaaa;

num[2]=1, j=1

求num[3]的时候, 比较a[j]和a[3]的值, 相等, $j=2 \leq (1+3)/2$

所以num[3]=2 j=2

求num[4]的是, 比较a[j]和a[4]的值, 相等 $j=3 > (1+4)/2$

那么就需要跳到 $j=nextn[j-1]=2$

所以num[4]=2, j=2

KMP 练习题

洛谷 3375 KMP匹配

Poj 2406 Power Strings

Poj 2752 Seek the Name

Hdu 剪花布条

BZOJ 1355 Radio Transmission

BZOJ 1511 Words

BZOJ 3620 似乎在梦中见过的样子

BZOJ 3942 Censoring

洛谷 2375 动物园

KMP算法主要用于求包含字符串匹配以及公共前后缀的问题。



KMP算法

KMP算法的时间复杂度是很高效的，接近于 $O(n)$ ，但是KMP算法只能用于单串匹配，对于多串匹配，他的时间复杂度仍然比较低，对于多串匹配，我们要考虑其他算法。



字典树

说明/提示

对于 40%的数据, $n \leq 1000, m \leq 2000$;

对于 70%的数据, $n \leq 10000, m \leq 20000$;

对于 100%的数据, $n \leq 10000, m \leq 100000$ 。

题目背景

XS中学化学竞赛组教练是一个酷爱炉石的人。

他会一边搓炉石一边点名以至于有一天他连续点到了某个同学两次，然后正好被路过的校长发现了然后就是一顿欧拉欧拉欧拉（详情请见已结束比赛CON900）。

题目描述

这之后校长任命你为特派探员，每天记录他的点名。校长会提供化学竞赛学生的人数和名单，而你需要告诉校长他有没有点错名。（为什么不直接不让他玩炉石。）

输入格式

第一行一个整数 n ，表示班上人数。接下来 n 行，每行一个字符串表示其名字（互不相同，且只含小写字母，长度不超过 50）。第 $n+2$ 行一个整数 m ，表示教练报的名字。接下来 m 行，每行一个字符串表示教练报的名字（只含小写字母，且长度不超过 50）。

输出格式

对于每个教练报的名字，输出一行。如果该名字正确且是第一次出现，输出“OK”，如果该名字错误，输出“WRONG”，如果该名字正确但不是第一次出现，输出“REPEAT”。（均不加引号）

字典树

如果下面的每一个字符串，都去和上面的每一个字符串用kmp算法匹配，那么时间复杂度为 $50 * n * m$ ，效率很低，所以对于多串匹配，我们应该找一个更加方便快捷的方法。

我们比较的时候完全可以一位一位的比较，先比较第一位，筛选出第一位相等的字符串，再比较第二位，从剩下的字符串中找到第二位相等，这样比较下去，剩下的字符串肯定越来越少，那么比较次数也会越来越少，效率会很高。

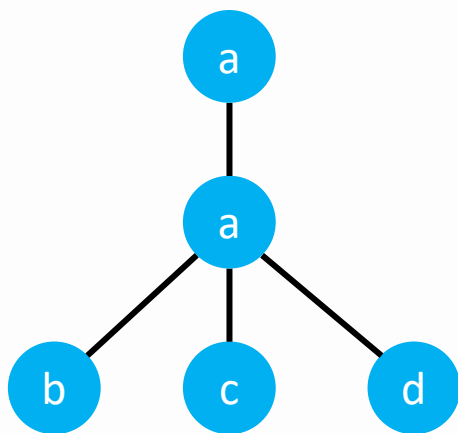
这样对于随机数据效率是很高的，但是如果数据中很多单词的前缀都是相同的，并且和目标字符串相同，那么效率并没有太大变化。

比如所有单词都是aaaaaaa...的形式，目标字符串也是aaaaaaa...的形式，那么我们应该怎么处理呢？

字典树

也就是说我们需要考虑很多前缀都相等的字符串，减少他们对效率的影响，既然他们的前缀相等，那么我们为什么不把他们的前缀合并呢？

比如aab, aac, aad，我们正常情况下存储是放入3个字符串中，但是实际上我们可以放入一棵树中存储。

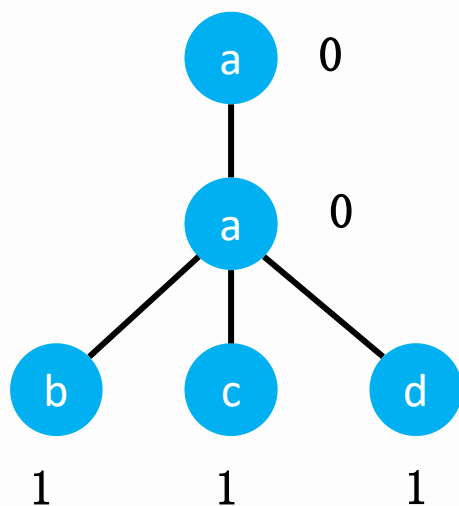


一条从根节点开始向下延伸的一条链就代表了一个字符串。这样存储就会省很多空间，相同前缀的字符就用一个前缀表示，这样查找也省时间。

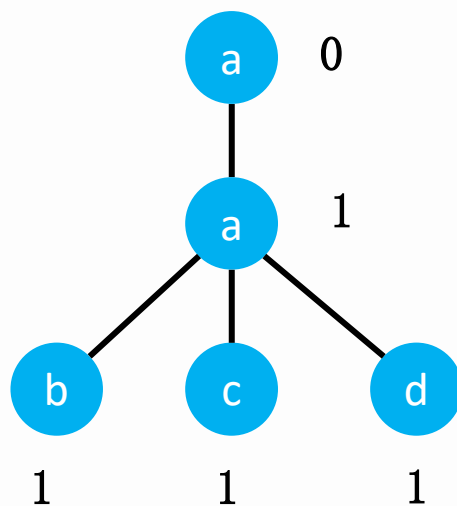
字典树

单词的长度并不是都相同的，如果该树形结构中还有一个单词aa，那么应该如何表示呢？我们会发现树的形态并不会发生改变，那么我们怎么才能直到里面到底哪些是单词呢？

其实也很简单，我们对于每一个节点，做一个标记，如果是单词结尾，那么就做上标记1，否则就是0：比如



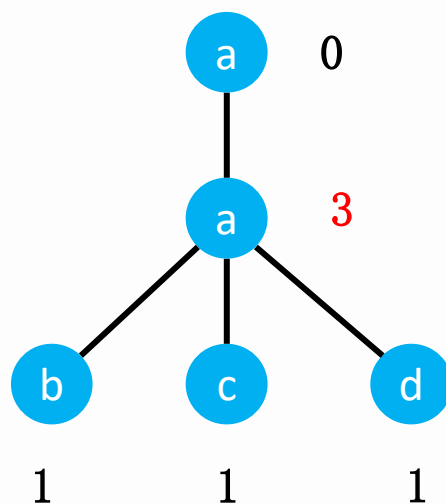
不加aa



加入aa

字典树

如果单词表中有3个相同的字符串aa，那么我们应该如何处理呢？



加入3个
aa

那么接下来我们就考虑一下如何来建这样一棵树。

字典树

首先，题目中说了只有小写字符，那么对于数上的每一个节点，最多有26个儿子(a—z对应0—25)。

假设所有的字符都没有公共前缀，那么最多有50*10000个节点，每个节点下面最多有26个节点。

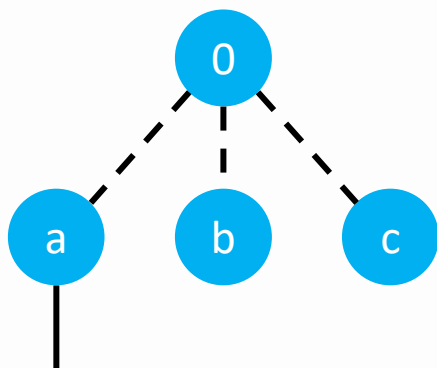
所以定义树的时候应该定义为：

```
const int maxn=500010;
struct node
{
    int next[26]; //只有26个小写字符，转换成0-25;
    int flag; //表示是否是完整单词
}tree[maxn];
```

字典树

既然我们要建一棵树，那么必须要有根节点，但是字符串的首字符都可能不相等，那么根节点用什么表示呢？

我们可以建一个虚拟的节点为根节点，假设就是0号节点，虚拟节点下面连着的就是每个字符串的第一个字符。



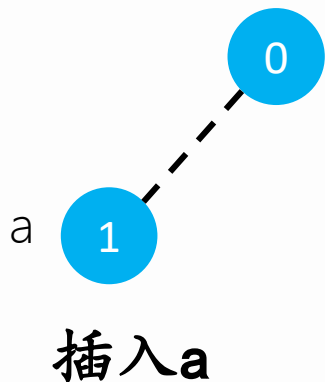
字典树

接下来考虑如何建树，也就是如何把一个字符串放入字典树中，最开始的时候字典树只有一个0号根节点。

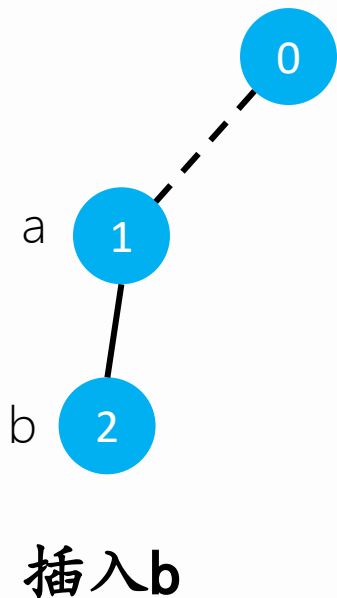
我们来模拟一下如何放入两个字符串abc和abd

(1) 每个节点下面第*i*个儿子实际上表示字符 $i + 'a' - 1$ ，首先我们看根节点下面第1个节点是否为空(如果为空，那么编号为0，不为空则编号为*k*)，如果为空，那么把该节点定义为1号节点，然后我们再跳到1号节点，再去看看1号节点的第2个节点(*b*)是否为空，为空，那么我们把这个节点定义为2号节点，接下来跳到2号节点，再去看看2号节点的第3个儿子是否为空，为空，那么我们把这个节点定义为3号节点。此时，单词便利结束，那么3号节点我们就对3号节点做一个标记，表示是单词的结尾。这样，我们就把字符串abc就放入字典树中了。

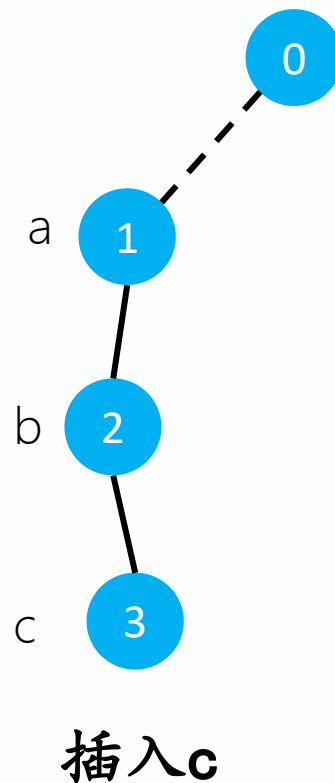
字典树



```
int t= 'a' - 'a' ;  
tree[0].next[t]=1
```



```
int t= 'b' - 'a' ;  
tree[1].next[t]=2
```



```
int t= 'c' - 'a' ;  
tree[2].next[t]=3  
tree[3].flag=1
```

字典树

再插入字符串abd

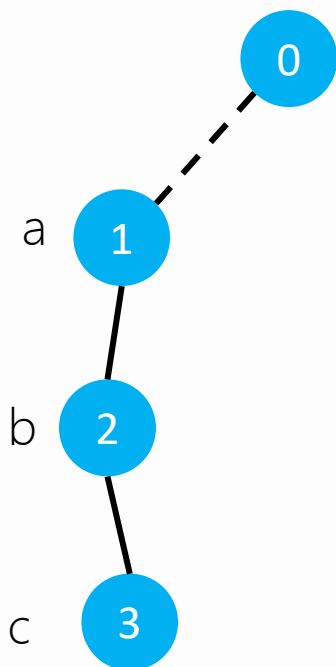
(1) 第一个字符a是根节点的第0个儿子，但是 `tree[0].next[0]` 已经存在了，编号为1，那么就不需要新建节点，直接跳转到1号节点。

(2) 第二个字符b是根节点的第1个儿子，但是 `tree[1].next[1]` 也已经存在了，编号为2，也不需要新建节点，直接跳转到2号节点

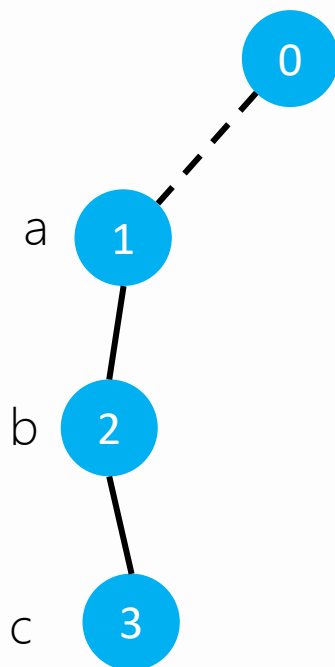
(3) 第三个字符串d是根节点的第4个儿子，`tree[2].next[4]` 并不存在，我们把这个节点定义为4号节点，并且让 `tree[2].next[4]=4`，做上单词结尾标记。

字典树

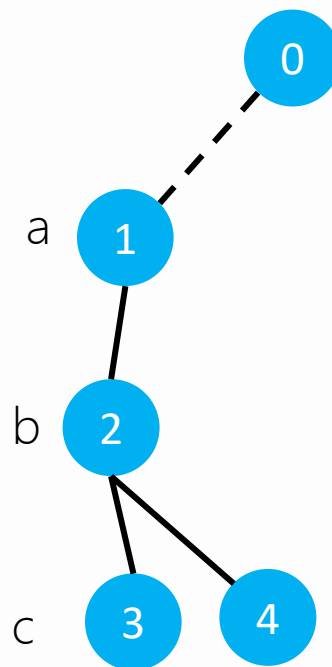
再插入字符串abd



字符a所在节点已经存在，不新增节点



字符b所在节点已经存在，不新增节点



字符d所在节点不存在，新增节点，编号为4

```
int t= 'd' - 'a' ;  
tree[2].next[t]=4;  
tree[4].flag=1
```

字典树

```
void build(int id,char *s)//注意这里用指针，只是传递字符的位置，一个字节，  
//如果传递整个字符串，时间复杂度为O(L)  
{  
    int l=strlen(s);//枚举字符串  
    for(int i=0;i<l;i++)  
    {  
        int t=s[i]-'a';//字符转换成0-25的数字  
        if(tree[id].next[t]==0)//如果找不到该单词，就在字典树中插入  
        {  
            k++;//新增一个节点，编号为k  
            tree[id].next[t]=k;//指针指向该节点  
        }  
        id=tree[id].next[t];//跳到下一个点  
    }  
    tree[id].flag=1;//flag为1表示单词的结尾标记  
    //如果从该点开始就不可以匹配了，那么就往下面创建该字符串  
    //如果该点可以匹配，那么就继续往下面匹配  
}  
for(int i=1;i<=n;i++)  
{  
    scanf("%s",str);  
    build(0,str);//0号节点表示根节点，把所有字符相连  
}
```

字典树

字典树建好之后，接下来就是用每一个目标字符串来字典树上进行匹配。

比如目标字符串为abdc, 字典树为abc, abd

(1) 0号节点为根节点, $\text{tree}[0].\text{next}[0]=1$, 表示该节点存在, 那么字符串中第一个字符串匹配到了。

(2) 跳转到1号节点, $\text{tree}[1].\text{next}[1]=2$, 表示该节点存在, 那么字符串中第二个字符串匹配到了。

(3) 跳转到2号节点, $\text{tree}[2].\text{next}[3]=4$, 表示该节点存在, 那么字符串中第三个字符串匹配到了。

(4) 跳转到4号节点, $\text{tree}[4].\text{next}[2]=0$, 该节点不存在, 那么说明最后一个字符没匹配到, 匹配失败。

字典树

字典树建好之后，接下来就是用每一个目标字符串来字典树上进行匹配。

比如目标字符串为ab，字典树为abc, abd

(1) 0号节点为根节点，`tree[0].next[0]=1`，表示该节点存在，那么字符串中第一个字符串匹配到了。

(2) 跳转到1号节点，`tree[1].next[1]=2`，表示该节点存在，那么字符串中第二个字符串匹配到了。

此时匹配结束，但是1号节点的flag为0，即1号节点并不是单词的结尾，所以并没有一个完整的单词为ab，所以匹配失败。

字典树

```
void find(int id,char *s)
{
    int l=strlen(s);
    for(int i=0;i<l;i++)
    {
        int t=s[i]-'a';
        if(tree[id].next[t]==0)//表示没有这个单词
        {
            printf("WRONG\n");//直接输出wrong
            return ;
        }
        id=tree[id].next[t];
    }
    ...
}

for(int i=1;i<=m;i++)
{
    scanf("%s",str);
    find(0,str);//从根节点开始查找该字符串
}
```

字典树的应用

一：求前缀：

- (1) 给定 m 个单词和 n 个前缀，分别求有多少个单词包含这些前缀
- (2) 给定 m 个字符串，判断是否存在一个单词是另一个单词的前缀

二：判断单词是否出现过

- (1) 给定一篇文章，再给定一些单词，判断这些单词是否在文章中出现过
- (2) 给定一些单词，再给定一篇文章，问这些单词是否可以完全构成这篇文章

三：树上异或值



Phone List

给定n个长度不超过10的数字串，问其中是否存在两个数字串S, T，使得S是T的前缀。 $N \leq 10^5$

输入：

2

3

911

97625999

91125426

5

113

12340

123440

12345

98346

输出：

YES

NO



Phone List

该题因为是求前缀，所以KMP没有任何意思，还不如暴力比较，如果用字符串hash，因为hash主要是判断两个字符串是否相等，如果要判断一个字符串和另一个字符串的前缀是否相等，还是只能一个一个比较，时间复杂度都是 $O(n^2)$ 。

所以我们考虑字典树，思路很简单，边建树边判断，对于当前新加入的字符串，在插入字典树的过程中，如果遇到了flag标记，就表示前面出现了单词是当前单词的前缀，那么就存在，如果所有单词插入完之后都不存在，那么就不存在。

L 语言

题目大意：给你n个单词，m篇文章，问该文章是否能被理解(如果该文章能被其中的某些单词完整的拼凑起来那么就认为可以被理解)。如果不能被完全理解，那么输出最多可以被理解的前缀

输入：

4 2

abc

abcd

def

efg

abcdef

abcdefg

输出：

6

7

L 语言

该题想到思路很简单，先把所有单词建一个字典树，然后再用文章去匹配，看一下最多可以匹配到哪个位置，关键是该题有一个细节很简单？假设字典树的一条路径上有多个单词，比如a, ab, abc, abcd，那么我该如何匹配？

有人可能觉得，那还不简单，能继续向下匹配肯定继续匹配噻？很明显样例就过不了。那我匹配到都是一个完整单词我就不匹配了，样例还是过不了，由于后面的不确定性，所以无论每次主观的判断匹配还是不匹配答案都是错误的，那应该怎么办？

很简单，都去尝试一遍。

L 语言

也就是说，先匹配一条链，把这条链上可能是单词结尾的地方都做一个标记。

11 12 13

然后把这些被标记过的点都当做起点继续匹配，把找到的一条链匹配完，继续标记可能出现的所有结尾，一直这样做下去，每次以当前被标记的点为起点，最终找到的可以匹配的最长的一定是答案。

L 语言

也就是说，先匹配一条链，把这条链上可能是单词结尾的地方都做一个标记。

11 12 13

然后把这些被标记过的点都当做起点继续匹配，把找到的一条链匹配完，继续标记可能出现的所有结尾，一直这样做下去，每次以当前被标记的点为起点，最终找到的可以匹配的最长的一定是答案。

最大异或值

给定N个数，从中选择两个数进行异或，问最大异或值是多少？

输入：

5

2 9 5 7 0

输出：

14

$N \leq 100000$, 值 $< 2^{31}$

最大异或值

首先，题目一看完暴力方法就想出来了，枚举异或呗，但是要AC呢？很容易想到一种思路，为了让异或的值最大，那么很明显要让最高位最大，对于异或来说，只有0和1两种，那么很明显就是如果原来这个数高位是1，那么我们就尽量先选择该位为0的，如果是0，我们就选则是1的，当然，如果不存在满足要求的那就没得选。为了优化，我们一般采用二分，但是我们又没有办法按每一位二分。

我们在模拟过程中会发现，比如高位需要一个1来异或，那么可以选的数的范围就减小了很多，那么下一位需要一个0，那么符合条件的数又少了了很多，这样一直一下，一定只有一个数满足条件。

关键是怎么来实现呢？怎样才能把前缀每一位的值相同的数放在一起呢？

01trie

其实可以用字典树来实现，当然这里是01trie，即树上的每一个结点表示数的每一位，这样我们就可以把前缀值相同的数放在一起了。比如101，要让异或值最大，我们首先最好给他匹配一个0，我们就在字典树中第一层(0结点下的那一层)查找是否存在有0的结点，如果存在，那么第一位找到了，如果不存在，那就只能找1了。接下来，我们需要找1，和前面一样的办法。每次找到答案之后把这条路径上找到的值按位累加上来到最后就是真实值。

思路很简单，代码也很简单。

01trie

```
struct node
{
    int son[2];
}tree[maxn<<5];// 每一个数可以拆分成31位，所以要*32
void build(int id,int x)
{
    for(int i=30;i>=0;i--)
    {
        int t=((x>>i)&1);// 数x的第i位的值
        if(tree[id].son[t]==0)// 如果不存在结点，新建一个
        {
            k++;
            tree[id].son[t]=k;
        }
        id=tree[id].son[t];
    }
}
```

01trie

```
int find(int id,int x)
{
    int ans=0;
    for(int i=30;i>=0;i--)
    {
        int t=!((x>>i)&1);//如果该位是1，那么再来一个数是0最好，如果是0，再来一个1最好
        if(tree[id].son[t])//如果有最优选项
        {
            id=tree[id].son[t];
            ans+=t<<i;//累加该位的值
        }
        else if(tree[id].son[t^1])//如果没有
        {
            id=tree[id].son[t^1];//如果没有也只能将就了
            ans+=(t^1)<<i;
        }
    }
    return ans;//把找到的答案返回回去
}
```

REBXOR

给定一个含 N 个元素的数组 A ，下标从 1 开始。请找出下面式子的最大值：

$$(A[l_1] \oplus A[l_1 + 1] \oplus \cdots \oplus A[r_1]) + (A[l_2] \oplus A[l_2 + 1] \oplus \cdots \oplus A[r_2])$$

其中 $1 \leq l_1 \leq r_1 < l_2 \leq r_2 \leq N$ 。式中 $x \oplus y$ 表示 x 和 y 的按位异或

输入：

5

1 2 3 1 2

输出：

6

$$n < 4 * 10^5 \quad a_i \leq 2^{32}$$

REBXOR

我们先思考如果只选择一个区间应该怎么做，也就是选择一个区间 $[l, r]$ ，让区间 $[l, r]$ 的所有数的异或值最大。

直接做很麻烦，我们可以把区间变成前缀，我们知道相同的两个数异或值为0，那么区间 $[l, r]$ 的异或，我们可以转换成两个前缀 $[1, l-1]$ 和 $[1, r]$ 的异或结果，因为都包含了 $[1, l-1]$ ，所以前面部分异或结果为0。

那么转换成前缀又该怎么做呢？ l 和 r 应该如何选择呢？我们前面学习的字典树只能做选择两个数的最大异或。那么我们就把前缀异或值看做一个数。即 $xor[i]$ 表示第1个数到第 i 个数的异或值，那么我们只需要找到两个数 $xor[i]$ ， $xor[j]$ 的异或值最大就可以了

这样就转换成一道字典树的裸题了，我们把 $xor[i]$ 的答案都放入字典树中。

REBXOR

我们先思考如果只选择一个区间应该怎么做，也就是选择一个区间 $[l, r]$ ，让区间 $[l, r]$ 的所有数的异或值最大。

直接做很麻烦，我们可以把区间变成前缀，我们知道相同的两个数异或值为0，那么区间 $[l, r]$ 的异或，我们可以转换成两个前缀 $[1, l-1]$ 和 $[1, r]$ 的异或结果，因为都包含了 $[1, l-1]$ ，所以前面部分异或结果为0。

那么转换成前缀又该怎么做呢？ l 和 r 应该如何选择呢？我们前面学习的字典树只能做选择两个数的最大异或。那么我们就把前缀异或值看做一个数。即 $xor[i]$ 表示第1个数到第 i 个数的异或值，那么我们只需要找到两个数 $xor[i]$ ， $xor[j]$ 的异或值最大就可以了

这样就转换成一道字典树的裸题了，我们把 $xor[i]$ 的答案都放入字典树中。

REBXOR

第一个问题解决了，但是还有一个问题，不是求一个区间最大，而是两个区间异或和之和最大，所以我们还不能先选一个最大的，再重剩下的里面再选一个最大的。

我们这个时候就可以考虑动态规划的思想了，我们可以开一个数组 $l[i]$ 表示从区间 $[1, i]$ 之间选择两个数的最大异或值。依次把每一个前缀加进来，每加入一个数，就求解新加入这个数和前面这些数的一个最大异或值，在和前一个的 $l[i-1]$ 取一个最大值，当做 $l[i]$ 的值。

那么对于后面的部分应该如何选择区间呢？其实我们可以看做后缀和，求解方法和前面求解 $l[i]$ 的方法一样，记做 $r[i]$ 。

我们再枚举 i ，把 $l[i]$ 和 $r[i]$ 相加，注意不能重叠

字典树练习题

洛谷 2580 于是他错误的点名开始了

洛谷 2292 L语言

洛谷 2922 秘密消息

洛谷 3061 疯狂的栅栏

洛谷 3879 阅读理解

Poj 3630 Phone List

LOJ 10050 The XOR Largest Pair

BZOJ 4260 REBXOR

洛谷 4551 最长异或路径

洛谷 3065 First

洛谷 4735 最大异或和

洛谷 3449 PAL-palindromes